COMPUTER ORGANIZATION AND ARCHITECTURE

ASSIGNMENT REPORT

RISC vs CISC | Harvard vs von Neumann
8085 and 8086 Microprocessors | Digital Logic Circuits
Read and Write Timing Diagrams

Submitted by: _____

Roll Number: _____

Course / Section: _____

Submitted to: _____

Department / Institution: _____

Date of submission: _____

A concept-focused report with architecture diagrams, comparison tables, logic equations, truth tables, and bus-cycle timing diagrams.

Abstract

This report studies foundational ideas in computer organization and digital logic. It compares Reduced Instruction Set Computer (RISC) and Complex Instruction Set Computer (CISC) design philosophies, contrasts Harvard and von Neumann memory organizations, and examines the Intel 8085 and 8086 microprocessors. For both processors, the report describes internal architecture, registers, instruction categories, and addressing modes. It then explains half adders, full adders, encoders, decoders, multiplexers, and common flip-flops using equations and truth tables. Finally, simplified memory-read and memory-write timing diagrams are presented for the 8085 and the 8086 in minimum mode. Emphasis is placed on how the datapath, control signals, and timing coordinate a valid transfer rather than on memorizing isolated definitions.

Objectives

- Distinguish RISC and CISC as ISA/design approaches without treating their common characteristics as absolute rules.
- Explain the data-path difference between Harvard and von Neumann organizations and the origin of the von Neumann bottleneck.
- Describe the architecture, registers, flags, instruction groups, and addressing modes of the 8085 and 8086.
- Derive and interpret the logic of adders, encoders, decoders, multiplexers, and flip-flops.
- Interpret read/write timing waveforms by relating T-states to multiplexed buses and active-low control signals.

How to use this report

Start with the architectural comparisons, then use the processor sections as a reference for the later timing diagrams. In the diagrams, a signal label ending in # (for example, RD#) means that the signal is active when low. Timing diagrams are simplified teaching representations of a standard no-wait-state memory cycle; small device-specific setup, hold, and propagation intervals are not drawn to scale.

Contents

Section	Coverage
1. Introduction to Computer Organization	Core terms and scope
2. RISC vs CISC	Definitions, traits, detailed comparison, modern nuance
3. Harvard vs von Neumann Architecture	Block diagrams, bottleneck, modified Harvard
4. Intel 8085 Microprocessor	Architecture, registers, flags, instructions, addressing modes
5. Intel 8086 Microprocessor	BIU/EU architecture, registers, instructions, addressing modes
6. 8085 vs 8086: Consolidated Comparison	Bus widths, memory, execution, instruction and system differences
7. Digital Logic Building Blocks	Adders, encoders, decoders, multiplexers, flip-flops
8. Read and Write Timing Diagrams	8085 and 8086 memory bus cycles
9. Conclusion	Consolidated observations
Appendix A. Quick Revision Sheet	Key equations and signals
References	Textbooks and processor manuals

Notation reminder Hexadecimal values are written with an H suffix (for example, 2050H). A bracketed operand such as [BX] denotes a memory operand in 8086 assembly notation. The exact assembler syntax may vary slightly across assemblers.

1. Introduction to Computer Organization

Computer organization explains how the functional parts of a computer are arranged and how they cooperate to execute instructions. Computer architecture describes the programmer-visible specification, including the instruction set, registers, data types, and memory model. Organization focuses more closely on implementation details such as buses, control signals, registers inside a datapath, timing, and the way instructions are fetched and executed.

1.1 Basic functional units

- Input and output units exchange information with the outside world.
- Memory stores instructions and data. Depending on the design, instruction and data storage may use separate or shared paths.
- The arithmetic logic unit (ALU) performs arithmetic and logical operations.
- Registers provide small, fast storage inside the processor for operands, addresses, intermediate results, and control state.
- The control unit decodes instructions and generates the sequencing and control signals needed to move data and select operations.
- Buses carry address information, data, and control/status information between processor, memory, and peripherals.

1.2 Why the topics are connected

RISC/CISC and Harvard/von Neumann describe different levels of design: instruction-set philosophy and memory-system organization. The 8085 and 8086 provide concrete processor examples in which the register set, bus width, instruction encoding, and control timing can be studied. Adders and storage elements are lower-level building blocks used to construct datapaths and control circuits. Timing diagrams connect the logical view to the actual sequence of electrical bus signals.

> **A useful distinction** The 8085 and 8086 are historical CISC-style processors, but neither should be treated as a perfect example of every CISC trait. Likewise, modern CPUs often combine features from multiple design traditions.

2. RISC vs CISC Comparison

2.1 Definitions

RISC stands for Reduced Instruction Set Computer. A RISC design traditionally favors a comparatively regular instruction set, simple instruction formats, a load/store model for memory operations, and implementation techniques that make pipelining predictable. The goal is not simply to minimize the number of instructions; it is to simplify or regularize instruction execution so that the implementation can achieve good performance and efficiency.

CISC stands for Complex Instruction Set Computer. A CISC design traditionally provides a richer set of instructions, more addressing forms, and sometimes instructions that perform multi-step tasks or operate directly on memory operands. Such instructions can make some programs compact, but their internal implementation can be more complex. Modern CISC processors may translate complex instructions into simpler internal micro-operations.

2.2 Typical characteristics

Feature	RISC tendency	CISC tendency
Instruction complexity	Many simple, regular operations	A broader set including complex operations
Instruction length	Often fixed length in a base ISA; some designs add compressed forms	Often variable length, as in classic x86
Memory operations	Commonly load/store: ALU works mainly on registers	Some instructions can combine memory access and computation
Addressing modes	Usually fewer and more regular	Often a larger variety
Control implementation	Historically often hardwired/regular control	Historically often microcoded for complex instructions
Pipelining	Regularity can simplify decode and scheduling	Variable length and complex semantics can make front-end work harder
Code size	May require more instructions for some tasks; compression can narrow the gap	Complex instructions can encode some tasks compactly
Compiler role	Compiler often combines simple operations effectively	Compiler chooses among a richer collection of instruction forms
Examples	ARM, RISC-V, classic MIPS designs	x86 family, including the 8086 lineage

2.3 Illustrative operation

Suppose a program must load two values, add them, and store the result. In a load/store RISC-style ISA, the likely sequence is: load the first value into a register, load the second value into another register, add the registers, and store the result. A CISC-style ISA may offer an instruction that uses a memory operand directly for some forms of the operation. Exact sequences depend on the specific ISA, data types, addressing mode, and compiler.

2.4 Advantages and trade-offs

Aspect	Potential RISC advantage	Potential CISC advantage / trade-off
Decode and pipeline	Regular formats can simplify instruction boundaries and control.	Complex/variable-length instructions may require more elaborate decoding.
Program size	Simple instructions can be efficient for regular workloads; compressed encodings may help.	Richer instructions can reduce the number of bytes or instructions for some operations.
Implementation	A regular ISA can make some implementations efficient.	Microcode or translation can preserve a rich programmer-visible ISA while changing its implementation.
Performance	Performance depends on the full pipeline, cache, compiler, and workload.	A single complex instruction is not automatically faster; internal work may still be several micro-operations.
Power and area	Simplicity can help, but not by itself guarantee lower power or area.	More complex decode may cost resources, while code density can reduce instruction-fetch traffic.

2.5 Important nuance

RISC and CISC are useful design categories, not a binary test of whether a processor is fast or slow. Many modern RISC processors have extensions and complex instructions; many modern x86 processors decode instructions into internal micro-operations that resemble simpler RISC-like operations. The visible instruction set and the internal execution engine are not necessarily the same thing.

Takeaway Compare instruction-set regularity, memory-operation style, encoding, and implementation consequences - not just the number of instructions in the ISA.

3. Harvard vs von Neumann Architecture

3.1 von Neumann organization

In a conventional von Neumann organization, instructions and data occupy a unified memory address space and use a shared memory system. The processor uses the same general memory path to fetch instruction bytes and to read or write data. This organization is flexible because program and data storage are managed within a unified model, but a shared path can limit how much instruction fetching and data transfer occur at once.

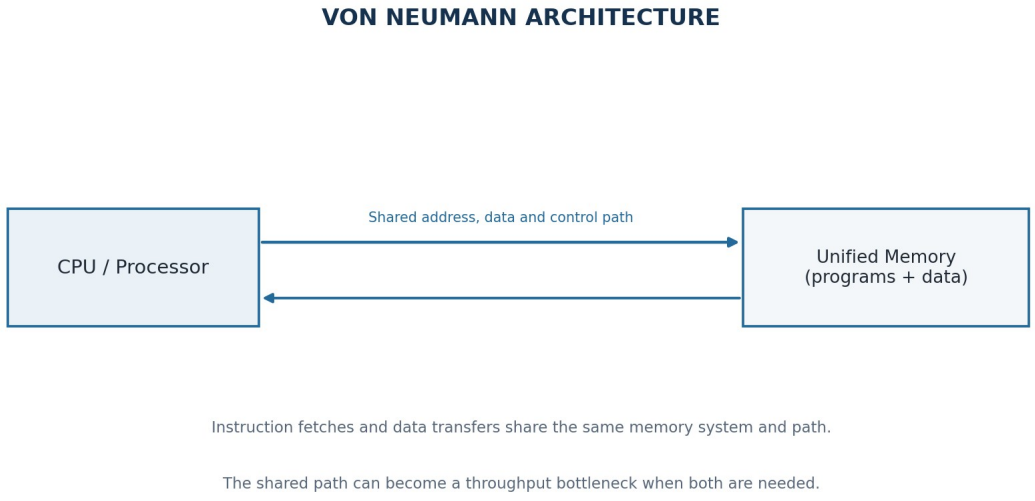


Figure 1. Simplified von Neumann organization: instructions and data share the memory path.

3.2 Harvard organization

In a Harvard organization, instruction memory and data memory are logically separate and typically have separate buses. The CPU may fetch the next instruction while simultaneously reading or writing data, provided the implementation and memory system support the concurrent accesses. The instruction and data spaces can also use different widths or technologies in some designs.

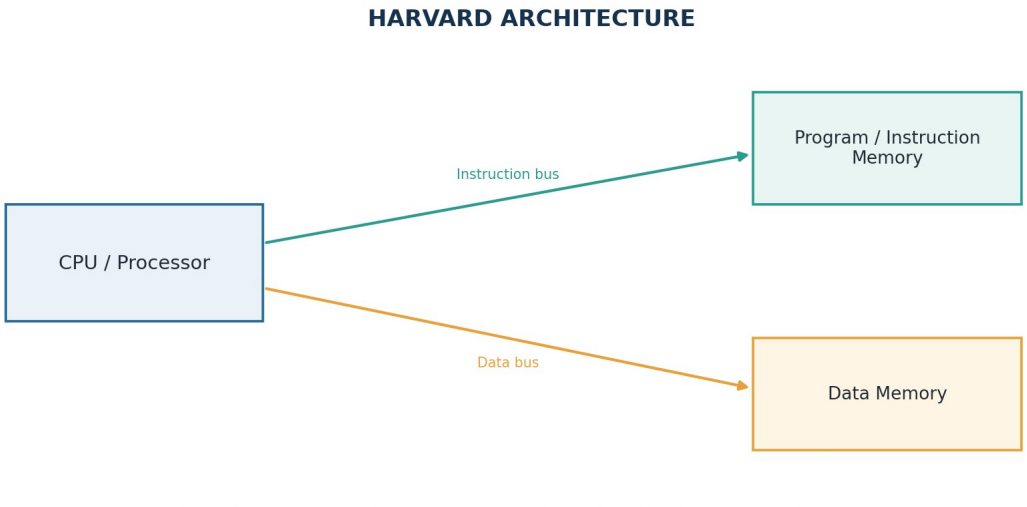


Figure 2. Simplified Harvard organization: separate instruction and data paths.

3.3 Detailed comparison

Property	von Neumann	Harvard
Instruction/data memory	Unified memory space in the conventional model	Separate instruction and data memories/spaces
Bus structure	Typically shared system path	Separate instruction and data paths
Concurrent fetch and data access	Can contend for the shared path	Can occur simultaneously when the hardware supports it
Memory flexibility	Program bytes can often be treated as data within the unified model	Instruction and data spaces may be accessed differently
Design complexity	Often simpler unified memory interface	May require separate interfaces and memory resources
Typical drawback	Shared-path bandwidth can create a bottleneck	Separate paths can increase design complexity and may restrict direct interchange between spaces
Examples / usage	Common conceptual model for general-purpose systems	Common in many microcontrollers and DSP-like designs; modified forms also occur in CPUs

3.4 The von Neumann bottleneck

The von Neumann bottleneck refers to the throughput limitation that may arise when instructions and data share a constrained path between the CPU and memory. Even if the ALU can perform work rapidly, the processor can stall when it cannot obtain instruction bytes or data quickly enough. Caches, wider buses, prefetching, and separate instruction/data caches can reduce this limitation, though the extent of improvement depends on the workload and implementation.

3.5 Modified Harvard organization

Many modern processors use a modified Harvard arrangement: main memory is unified at the software/system level, while the processor has separate instruction and data caches or separate internal paths. This combines a flexible unified address space with the ability to fetch instructions and access data in parallel in common cases. Therefore, it is inaccurate to say that every modern computer is purely Harvard or purely von Neumann at every level.

> **Key comparison** Harvard vs von Neumann concerns the organization of instruction and data memory paths. RISC vs CISC concerns the instruction-set/design approach. These are separate dimensions and should not be conflated.

4. Intel 8085 Microprocessor

4.1 Overview

The Intel 8085 is an 8-bit microprocessor introduced in the late 1970s. Its ALU and general data operations are primarily 8-bit, its external data bus is 8-bit, and its address bus is 16-bit, allowing it to address up to 2^16 = 65,536 byte locations (64 KiB) in the standard memory-address space. The lower eight address bits share the AD7-AD0 pins with the data bus, so these pins are multiplexed.

4.2 Internal architecture

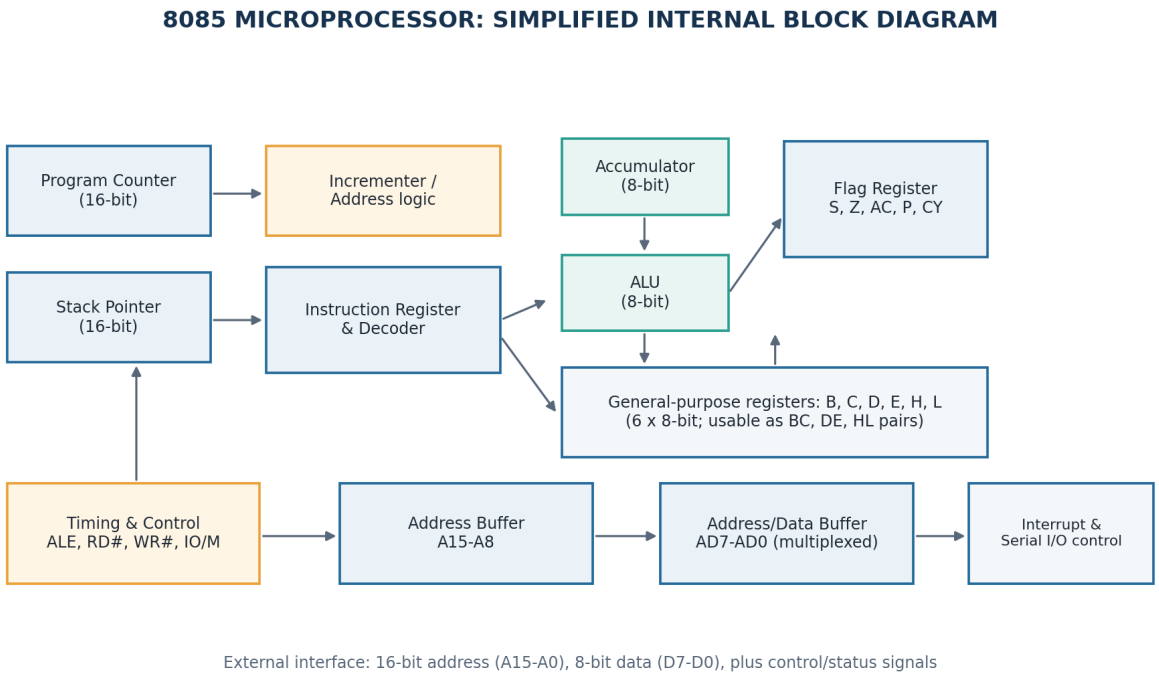


Figure 3. Simplified 8085 block diagram; functional blocks and buses are shown, not transistor-level circuitry.

The main architectural blocks are summarized below.

Block	Function
Accumulator (A)	An 8-bit register used as a primary ALU operand and as the destination/source for many arithmetic and logical instructions.
ALU	Performs arithmetic and logical operations on 8-bit operands and updates the relevant flags.
General-purpose registers	B, C, D, E, H, and L are 8-bit registers. They may also be used as register pairs BC, DE, and HL for 16-bit operations.
Program Counter (PC)	A 16-bit register holding the address of the next instruction byte to be fetched, subject to control-flow changes.
Stack Pointer (SP)	A 16-bit register identifying the top of the stack in memory. The stack typically grows toward lower addresses on push operations.
Instruction register and decoder	Hold/decode the current opcode and help generate the sequence of control actions for that instruction.

Block	Function
Timing and control	Generates and coordinates machine-cycle control signals such as ALE, RD#, WR#, and IO/M.
Address/data buffers	Present the address and data signals at the external pins. AD7-AD0 are multiplexed; A15-A8 carry the high-order address.
Interrupt / serial I/O control	Supports hardware interrupt inputs and serial input/output features such as SID and SOD.

4.3 Buses and key signals

Signal / bus	Meaning
A15-A8	High-order 8 address bits; dedicated address pins.
AD7-AD0	Multiplexed low-order address/data pins. During T1 they carry A7-A0; later they are reused for data.
ALE	Address Latch Enable. A pulse indicates that the low-order address is present on AD7-AD0 and can be latched externally.
RD#	Active-low read control. When asserted, the selected device is expected to provide data to the processor.
WR#	Active-low write control. When asserted, the selected device is instructed to accept data from the processor.
IO/M	Distinguishes an I/O machine cycle from a memory machine cycle; IO/M = 0 denotes a memory cycle in the common 8085 convention.
READY	Allows slower devices to request wait states so the processor can extend a machine cycle.
HOLD / HLDA	Used for bus request/acknowledgment to permit another bus master to take control, subject to the processor's bus-hold behavior.

4.4 Registers and flags

The six general-purpose registers can be used individually for byte values or in pairs for selected 16-bit operations. The HL pair is especially important because the symbol M in many 8085 instructions means the memory byte whose address is held in HL.

Register / pair	Size	Typical role
A (accumulator)	8-bit	Primary operand/result register for many ALU operations.
B, C, D, E, H, L	Each 8-bit	General-purpose byte registers; BC, DE and HL pairings are also supported.
PC	16-bit	Address of the next instruction byte, unless altered by branching, calls, returns, or interrupts.
SP	16-bit	Stack pointer.
Flags	5 condition flags used	S, Z, AC, P, CY record selected results of ALU operations.

Flag	Meaning	Condition / interpretation
S - Sign	Sign of 8-bit result	Set when bit 7 of the result is 1.
Z - Zero	Zero result	Set when the result is 00H.
AC - Auxiliary Carry	Carry between bit 3 and bit 4	Used especially by decimal-adjust operations.
P - Parity	Parity of result	Set when the result contains an even number of 1 bits.
CY - Carry	Carry out / borrow indication	Set by relevant arithmetic operations when a carry out occurs; for subtraction, the flag convention indicates borrow.

4.5 Instruction-set organization

8085 instructions can be grouped by what they do. The examples below are representative rather than an exhaustive opcode catalogue.

Category	Example instructions	Purpose / example interpretation
Data transfer	MOV A,B; MVI A,32H; LXI H,2050H; LDA 2050H; STA 2050H; LDAX B; STAX D; XCHG; PUSH B; POP H; IN 80H; OUT 81H	Copy values between registers, memory, stack, and I/O ports; load 16-bit register pairs; exchange HL and DE.
Arithmetic	ADD B; ADC M; SUB C; SBB D; INR A; DCR M; INX H; DCX SP; DAD B; DAA	Add/subtract with or without carry/borrow, increment/decrement, 16-bit pair arithmetic, and decimal adjustment.
Logical / rotate	ANA B; XRA M; ORA C; CMP D; CPI 20H; CMA; RLC; RRC; RAL; RAR; STC; CMC	Bitwise AND/XOR/OR, compare, complement, rotate, and carry-flag manipulation.
Branch / control flow	JMP 2050H; JZ 2050H; JC 2050H; CALL 2100H; RET; RST 1; PCHL	Unconditional and conditional jumps, subroutine calls/returns, restart vectors, and jump through HL.
Machine / interrupt control	NOP; HLT; DI; EI; SIM; RIM	No operation, halt, interrupt enable/disable, and serial/interrupt-mask related functions.

4.6 8085 addressing modes

An addressing mode specifies how an instruction obtains its operand. In the 8085, many memory instructions use the HL pair indirectly; M is an operand notation, not a physical register.

Mode	How operand is specified	Example	Interpretation
Immediate	Constant is part of the instruction bytes.	MVI A,32H	Load 32H directly into A.
Register	Operand is in a named register.	MOV A,B	Copy B to A.
Direct	A 16-bit memory address is explicit in the instruction.	LDA 2050H	Load A from memory address 2050H.
Register-indirect	A register pair supplies the memory address.	MOV A,M; LDAX B	MOV A,M reads address in HL; LDAX B reads address in BC.
Implied / inherent	Operand is implied by the opcode.	CMA; RAL; STC	Complement A, rotate A through carry, or set carry; the operand need not be written.

5. Intel 8086 Microprocessor

5.1 Overview

The Intel 8086 is a 16-bit microprocessor with a 16-bit external data bus and a 20-bit address bus. A 20-bit byte address selects up to 2^20 = 1,048,576 bytes (1 MiB) of physical address space. The 8086 uses segmented addressing: a segment register and an offset are combined to form a physical address. Its architecture is divided into a Bus Interface Unit (BIU) and an Execution Unit (EU), allowing instruction fetching to overlap with execution in many cases.

5.2 BIU and EU architecture

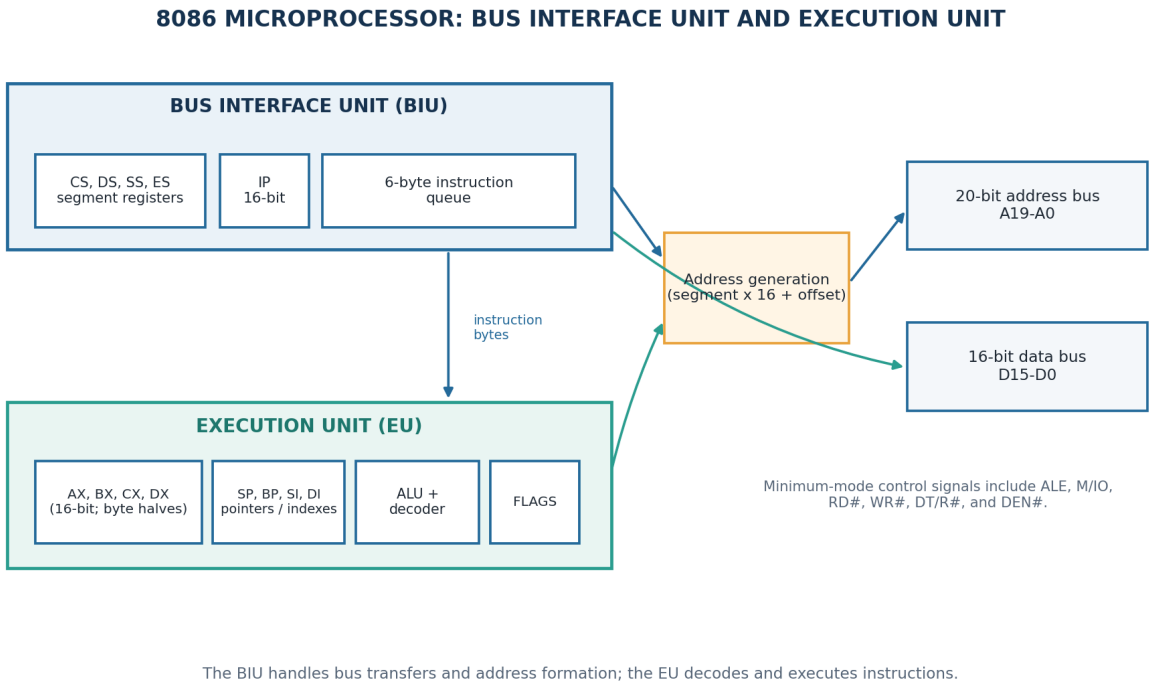


Figure 4. Simplified 8086 architecture, showing the BIU/EU split and the instruction queue.

Unit	Main contents	Main responsibilities
Bus Interface Unit (BIU)	CS, DS, SS, ES segment registers; Instruction Pointer (IP); 6-byte instruction queue; address-generation and bus-control logic.	Fetches instruction bytes, forms physical addresses, and carries out memory/I/O bus transfers.
Execution Unit (EU)	ALU; instruction decoder/control; AX, BX, CX, DX; SP, BP, SI, DI; FLAGS.	Takes instruction bytes from the queue, decodes and executes them, operates on registers/operands, and updates flags.

5.3 Segmented addressing

The physical address is calculated from a 16-bit segment value and a 16-bit offset using the 8086 rule:

Physical address Physical address = (segment value x 16) + offset = (segment value << 4) + offset.

For example, if CS = 1234H and IP = 0100H, the corresponding instruction-fetch physical address is 12340H + 00100H = 12440H. Segment:offset pairs can overlap: multiple pairs may refer to the same physical byte because the segment is shifted left by four bits before adding the offset. The address calculation has a 20-bit result, so the physical address space is limited to 1 MiB.

5.4 Registers and flags

Register group	Members	Main use
General-purpose	AX, BX, CX, DX	16-bit registers; each can be split into high and low 8-bit halves (AH/AL, BH/BL, CH/CL, DH/DL). Typical uses include arithmetic, base addressing, counts, and data.
Pointer and index	SP, BP, SI, DI	Stack pointer, base pointer, source index, destination index. Commonly used in stack, array, and string operations.
Segment	CS, DS, SS, ES	Code, data, stack, and extra segments; selected with offsets to create physical addresses.
Instruction Pointer	IP	Offset of the next instruction byte within the code segment.
FLAGS	CF, PF, AF, ZF, SF, TF, IF, DF, OF	Status flags report arithmetic/logic conditions; control flags govern trap behavior, interrupts, and string direction.

Key 8086 flags: CF (carry), PF (even parity of low result byte), AF (auxiliary carry), ZF (zero), SF (sign), OF (signed overflow), TF (single-step trap), IF (maskable interrupt enable), and DF (string direction). A flag is not updated by every instruction; its behavior is instruction-specific.

5.5 Instruction queue and overlap

The BIU fetches instruction bytes and places them in a 6-byte queue while the EU executes previously fetched instructions. This is a simple form of fetch/execute overlap, not the same as the deep speculative pipelines of modern processors. A control-transfer instruction such as a jump or call may change the next instruction address and cause the queue to be discarded or refilled from the new location.

5.6 Instruction-set organization

Category	Representative 8086 instructions	Role
Data transfer	MOV, PUSH, POP, XCHG, IN, OUT, LEA, LDS, LES, LAHF, SAHF, PUSHF, POPF	Move data among registers, memory, stack, segment-related pointers, and I/O; load effective address; save/restore flags.
Arithmetic	ADD, ADC, SUB, SBB, INC, DEC, CMP, MUL, IMUL, DIV, IDIV, NEG, DAA, DAS, AAA, AAS, AAM, AAD	Integer arithmetic, comparisons, multiplication/division, negation, and decimal/ASCII adjustment operations.
Logical / bit operations	AND, OR, XOR, NOT, TEST, SHL/SAL, SHR, SAR, ROL, ROR, RCL, RCR	Bitwise logic, bit testing, shifts, and rotations.
String operations	MOVSB/MOVSW, CMPSB/CMPSW, SCASB/SCASW, LODSB/LODSW, STOSB/STOSW, REP prefixes	Operate on byte/word sequences, generally using SI/DI and direction/count controls.
Control transfer	JMP, CALL, RET, conditional Jcc instructions, LOOP, JCXZ	Branch, call and return from procedures, and implement loop/counter-controlled flow.

Category	Representative 8086 instructions	Role
Interrupt and processor control	INT, INTO, IRET, CLC, STC, CLI, STI, CLD, STD, HLT, WAIT, LOCK prefix	Software interrupts/return, flag controls, halt/wait, and bus-lock behavior in supported contexts.

5.7 8086 addressing modes

The 8086 has more extensive operand-addressing forms than the 8085. The address expression inside square brackets specifies an effective offset; the default segment is normally DS, while BP-based effective addresses normally use SS unless a segment override is applied.

Mode	Example	Explanation
Immediate	MOV AX,1234H	The constant 1234H is encoded in the instruction.
Register	MOV AX,BX	Source and destination are registers.
Direct memory	MOV AL,[1234H]	The displacement gives the effective offset directly; default data segment is typically DS.
Register-indirect	MOV AX,[BX]	BX holds the effective offset of the memory operand.
Based + displacement	MOV AX,[BP+06H]	BP plus a displacement forms the effective offset; default segment is SS.
Indexed	MOV AX,[SI]	SI supplies the effective offset; often useful for array/string data.
Based-indexed	MOV AX,[BX+SI]	A legal base register and index register are added.
Based-indexed + displacement	MOV AX,[BP+DI+10H]	Base, index, and displacement combine to form the effective offset.
Implied / string operand	LODSB; STOSW; MUL BL	Some instructions imply operands such as AL/AX, SI/DI, or the accumulator.

5.8 Addressing restrictions worth remembering

- The usual 8086 base registers for effective-address formation are BX and BP; index registers are SI and DI.
- Legal combinations include BX+SI, BX+DI, BP+SI, and BP+DI, optionally with a displacement. Arbitrary combinations such as AX+CX are not valid 8086 memory effective-address expressions.
- BP-based effective addresses default to SS; most other ordinary memory effective addresses default to DS. A segment override can change the segment used.
- The 8086 generally does not support a normal MOV instruction with both source and destination as memory operands; one side must usually be a register or immediate value.

8086 memory operand reminder MOV AX,[BX] means read a 16-bit word at the effective address held in BX. MOV AX,BX transfers the register contents and does not access memory.

6. 8085 vs 8086: Consolidated Comparison

Both processors are early Intel microprocessors, but the 8086 extends the data width, address space, register set, and instruction capabilities. The comparison below uses the standard 8085 and original 8086 models; later family members and board-level implementations can differ.

Feature	8085	8086
Basic word/data processing width	8-bit	16-bit
External data bus	8 bits	16 bits
Address bus	16 bits	20 bits
Maximum byte-addressable physical memory	64 KiB (2^16 bytes)	1 MiB (2^20 bytes)
Low address/data pins	AD7-AD0 carry low address during T1 and data later	AD15-AD0 carry low 16 address bits during T1 and data later
High address pins	A15-A8 dedicated address outputs	A19-A16 multiplexed with status; BHE# shares a pin with status
Internal organization	Accumulator/ALU, register array, instruction decode, control/timing	BIU and EU with segmented address generation and 6-byte prefetch queue
General registers	A, B, C, D, E, H, L; selected register pairs	AX, BX, CX, DX with byte halves; SP, BP, SI, DI; segment registers
Program/stack state	16-bit PC and SP	IP plus CS and other segment registers; 16-bit SP and SS
Memory addressing model	16-bit address space without the 8086 segment:offset mechanism	Segment:offset calculation creates a 20-bit physical address
Instruction/data categories	Byte-oriented arithmetic, register pairs, conditional control flow, I/O, interrupt and serial controls	16-bit arithmetic plus richer addressing, string operations, multiplication/division and more control forms
Instruction queue / overlap	No comparable 6-byte instruction prefetch queue	6-byte instruction queue in the BIU; fetch/execute overlap when possible
Typical clocking context	Clock rate varies by chip variant/system implementation	Clock rate varies by 8086 version/system implementation
Typical system-level consequence	Simpler 8-bit interfaces; smaller direct address space	Higher data bandwidth per bus transfer and a larger directly addressable space, with more complex interfacing

6.1 Why the 8086 is not simply “twice as fast”

A 16-bit data bus can transfer a 16-bit word in one bus transfer when alignment and bus conditions permit, while an 8-bit 8085 bus may need multiple transfers for a multi-byte value. However, total performance also depends on clock speed, instruction timing, memory wait states, peripheral speed, software, and the exact instruction sequence. Width alone does not determine execution speed.

6.2 Design implications

- The 8085 often provides a more straightforward starting point for learning the fetch-decode-execute cycle and basic bus control.
- The 8086 introduces important system-level concepts: multiplexed 16-bit address/data pins, segment:offset addressing, the BIU/EU split, and instruction prefetch.
- Both processors require external memory and I/O interface logic in typical systems. Bus width and control timing affect how such devices are connected.

7. Digital Logic Building Blocks

Digital circuits are often classified as combinational or sequential. A combinational circuit produces outputs from the current inputs only (for example, an adder or multiplexer). A sequential circuit also stores state, so its output depends on current inputs and previous state (for example, a flip-flop). These building blocks appear in ALUs, register files, address logic, control units, and memory interfaces.

7.1 Adders

7.1.1 Half adder

A half adder adds two one-bit inputs A and B. It produces a one-bit sum S and a carry C. Because it has no carry-in input, it is suitable for the least-significant bit of a simple addition but cannot by itself represent a carry from a previous stage.

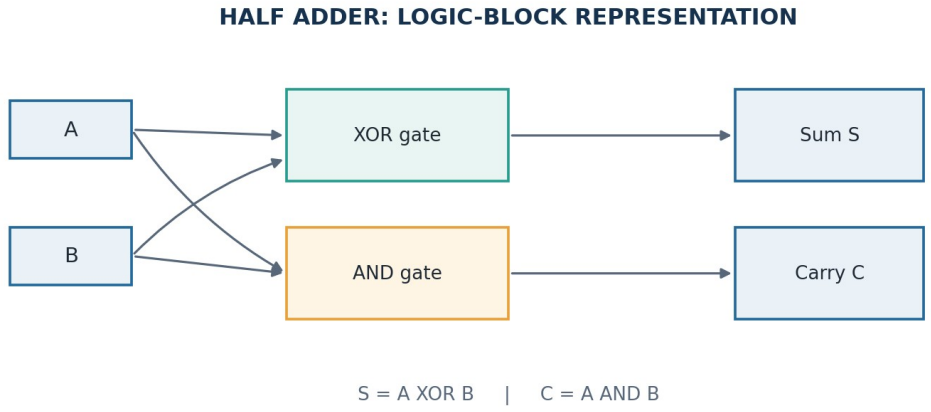


Figure 5. A half adder uses XOR for sum and AND for carry.

A	B	Sum S	Carry C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half-adder equations S = A XOR B; C = A AND B.

7.1.2 Full adder

A full adder adds A, B, and a carry-in Cin. It produces a sum S and carry-out Cout. Full adders can be chained to construct multi-bit adders such as a ripple-carry adder.

A	B	Cin	Sum S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

A	B	Cin	Sum S	Cout
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full-adder equations S = A XOR B XOR Cin; Cout = AB + ACin + BCin = AB + Cin(A XOR B).

7.1.3 Ripple-carry adder

A ripple-carry adder connects the carry-out of bit i to the carry-in of bit i+1. For an n-bit addition, the carry may have to propagate through several stages, so the worst-case carry delay grows with bit width. Faster adders reduce this carry-propagation delay using more elaborate logic, such as carry-lookahead structures.

7.2 Encoders

An encoder maps one of several active input lines into a binary code. A basic 4-to-2 encoder assumes at most one input is active at a time. When multiple inputs can be active, a priority encoder defines which input wins and commonly provides a valid output to distinguish “no input active” from input D0 encoding as 00.

D3	D2	D1	D0	Y1 Y0	Valid	Meaning
1	X	X	X	11	1	D3 has highest priority
0	1	X	X	10	1	D2 is highest active input
0	0	1	X	01	1	D1 is highest active input
0	0	0	1	00	1	Only D0 is active
0	0	0	0	00	0	No active input

In the table, X means “don’t care”: the value does not affect the result because a higher-priority input is already active. A basic encoder’s simple equations are valid only under its one-hot input assumption; priority behavior must be designed explicitly.

7.3 Decoders

A decoder activates one output corresponding to a binary input code. A 2-to-4 decoder has two input bits and four outputs. With enable E = 1, exactly one output is asserted for each input combination; with E = 0, all outputs are inactive in a typical active-high design.

Enable E	A1	A0	Y0	Y1	Y2	Y3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0

Enable E	A1	A0	Y0	Y1	Y2	Y3
1	1	0	0	0	1	0
1	1	1	0	0	0	1

2-to-4 decoder equations Y0 = E * NOT(A1) * NOT(A0); Y1 = E * NOT(A1) * A0; Y2 = E * A1 * NOT(A0); Y3 = E * A1 * A0.

Decoders are commonly used for memory chip selection, instruction decoding, and selecting one output line from an address. Active-low decoder outputs also exist; always check the polarity shown by the device symbol or datasheet.

7.4 Multiplexers (MUX)

A multiplexer selects one of several data inputs and connects the selected input to a single output. A 4-to-1 MUX has four data inputs I0-I3 and two select inputs S1,S0. It is a data selector; the select bits specify which input reaches Y.

S1	S0	Output Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

4-to-1 MUX equation Y = NOT(S1)NOT(S0)I0 + NOT(S1)S0I1 + S1NOT(S0)I2 + S1S0I3.

Multiplexers can route operands to an ALU, select data sources within a datapath, and implement Boolean functions. A bus system that permits several devices to share wires needs a way to choose which source drives those wires at a particular time; multiplexing is one common part of such designs.

7.5 Flip-flops

A flip-flop is a one-bit sequential storage element. In an edge-triggered flip-flop, the stored output normally updates at a specified clock edge. The next-state tables below assume the conventional active-high SR inputs and an edge-triggered interpretation; actual integrated circuits may provide asynchronous preset/clear or different input polarities.

7.5.1 SR flip-flop

S	R	Next state Q(next)	Meaning
0	0	Q	Hold previous state
0	1	0	Reset
1	0	1	Set
1	1	Invalid / not allowed	Forbidden condition in the conventional SR form

The SR form is intuitive but has an invalid input combination when both set and reset are asserted in the basic implementation. This is one reason the JK and D forms are often more convenient in digital systems.

7.5.2 JK flip-flop

J	K	Next state Q(next)	Meaning
0	0	Q	Hold
0	1	0	Reset
1	0	1	Set
1	1	NOT Q	Toggle

The JK form removes the forbidden SR case in its ideal truth table. When J = K = 1, the output toggles at the active clock edge. In practical level-triggered designs, timing details such as race-around must be addressed; edge-triggered or master-slave designs avoid the classic problem.

7.5.3 D flip-flop

D	Next state Q(next)	Meaning
0	0	Store 0 at active clock edge
1	1	Store 1 at active clock edge

For a D flip-flop, Q(next) = D at the active clock edge. The D input avoids the SR invalid combination and is widely used to build registers, pipeline stages, and synchronously updated state.

7.5.4 T flip-flop

T	Next state Q(next)	Meaning
0	Q	Hold
1	NOT Q	Toggle

A T flip-flop toggles when T = 1 and holds when T = 0. It is useful for counters and divide-by-two behavior. In many designs, T behavior can be implemented from a JK flip-flop by tying J and K together, or from a D flip-flop with D = Q XOR T.

7.6 Combinational vs sequential circuit summary

Circuit	Combinational / sequential	Stores state?	Typical use
Adder	Combinational	No	Arithmetic datapath
Encoder / decoder	Combinational	No	Code conversion and selection
Multiplexer	Combinational	No	Routing/selecting data sources
Flip-flop	Sequential	Yes, 1 bit per element	Registers, flags, counters, state machines

8. Read and Write Timing Diagrams

8.1 How to read a timing diagram

A timing diagram shows how signals change over time. The clock divides a bus cycle into time states. A signal drawn high is logic 1 and a signal drawn low is logic 0. A control signal marked with # is active-low: the action is requested while the waveform is low. A bus is not always a simple 0/1 waveform; it may carry an address, then data, or be driven by a different device at different times.

- Address phase: the processor presents the address that identifies the memory location or I/O port.
- Address latching: ALE or another strobe indicates when multiplexed address pins hold a valid address. An external latch preserves the address after those pins are reused for data.
- Control phase: RD# or WR# identifies the type of transfer and its active interval.
- Data phase: the device selected by the address either drives read data or accepts write data. Setup/hold requirements must be met around the sampling/acceptance interval.
- Wait states: if the device cannot respond quickly enough, READY-related logic can extend the bus cycle. The diagrams below omit wait states to make the core sequence clearer.

Scope of the diagrams The 8085 diagrams show a memory cycle (IO/M = 0). The 8086 diagrams show a minimum-mode memory cycle (M/IO = 1). Maximum-mode 8086 systems use an external bus controller and different command signalling; their waveforms are not identical to the minimum-mode diagrams.

8.2 8085 memory-read cycle

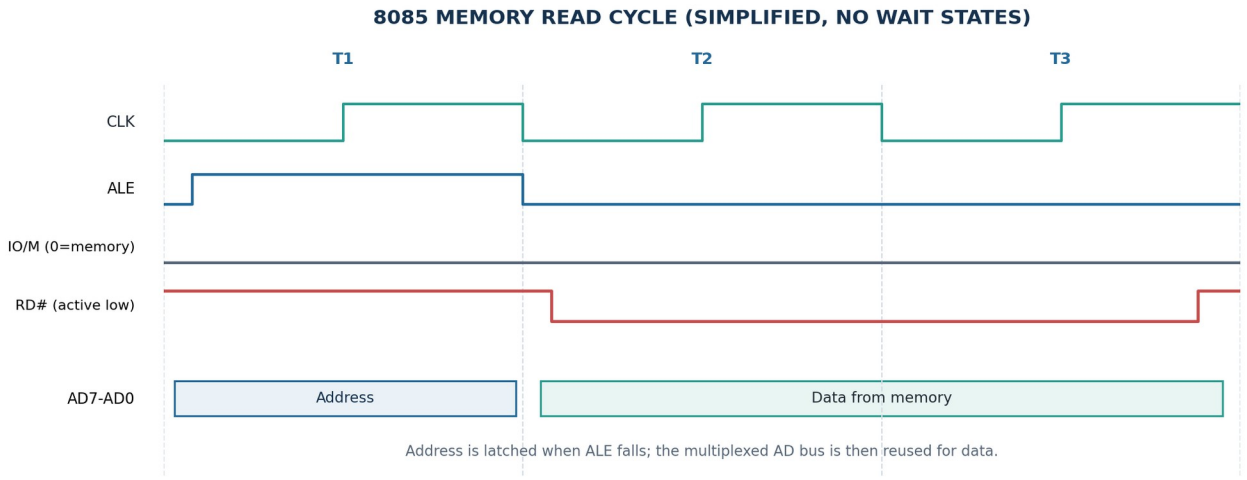


Figure 6. Simplified 8085 memory-read cycle across T1-T3.

During T1, A15-A8 and AD7-AD0 present the high- and low-order address portions. ALE goes high to tell an external latch that AD7-AD0 contains the low-order address. When ALE falls, the latch preserves A7-A0 even though the processor reuses AD7-AD0 for data. IO/M is low because the cycle accesses memory. RD# is asserted low during the read portion; the addressed memory device drives the data byte onto AD7-AD0, and the CPU samples the valid value before the cycle completes. RD# returns high to terminate the read strobe.

8.3 8085 memory-write cycle

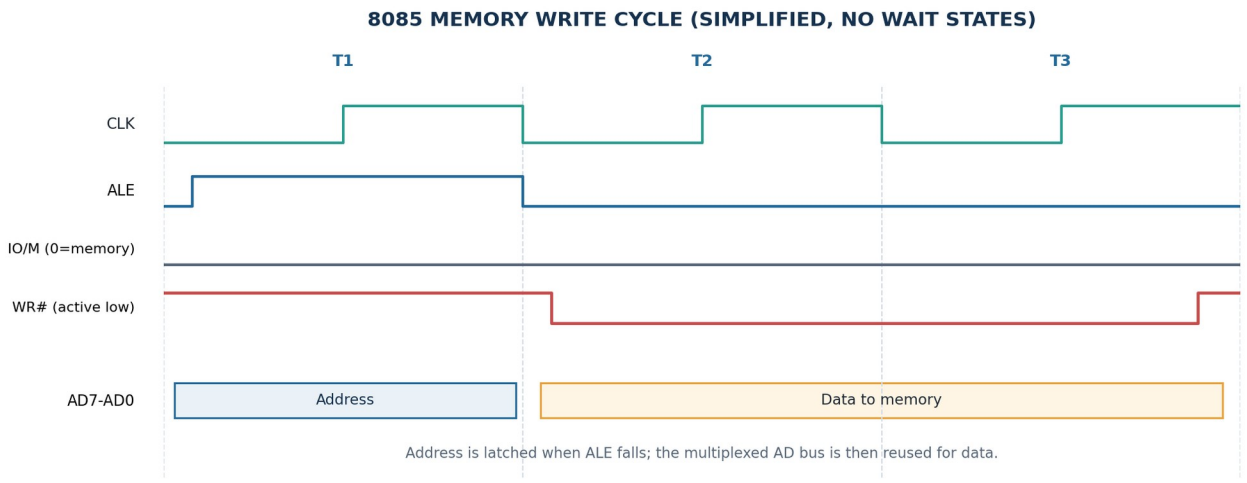


Figure 7. Simplified 8085 memory-write cycle across T1-T3.

The address phase and ALE behavior are similar to a read cycle. After the low-order address has been latched, the processor places the byte to be written on AD7-AD0. WR# goes low during the write portion so that the selected memory device captures the data according to its timing requirements. IO/M remains low for a memory operation. The processor must keep the write data stable for the required setup and hold interval around the receiving device's write timing.

8.4 8086 minimum-mode memory-read cycle

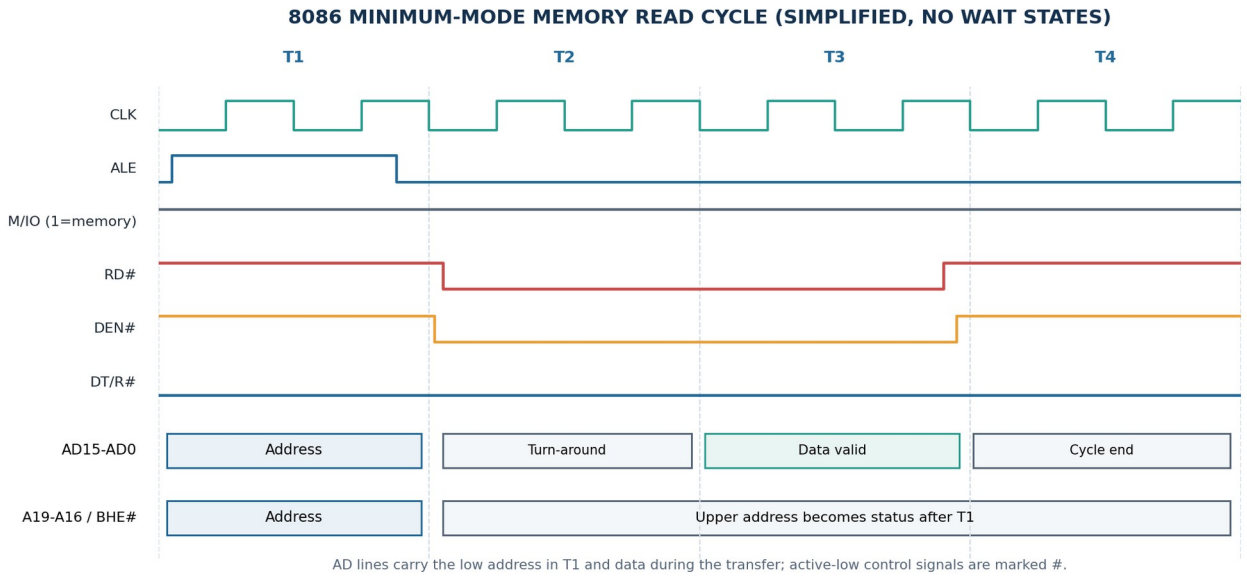


Figure 8. Simplified 8086 minimum-mode memory-read cycle across T1-T4.

In T1, AD15-AD0 carry the low 16 bits of the address and A19-A16/BHE# provide the upper address and byte-bank selection information. ALE pulses so that external latches can preserve the address while the AD pins are reused. M/IO is high for a memory cycle in the standard 8086 minimum-mode signal convention. During the read phase, RD# is active low and DEN# enables the external data transceiver. DT/R# is low to indicate that the processor is receiving data. The memory subsystem drives the data bus, and the processor accepts valid

data before the cycle ends. The AD bus can be high impedance during the transition before the memory data becomes valid.

8.5 8086 minimum-mode memory-write cycle

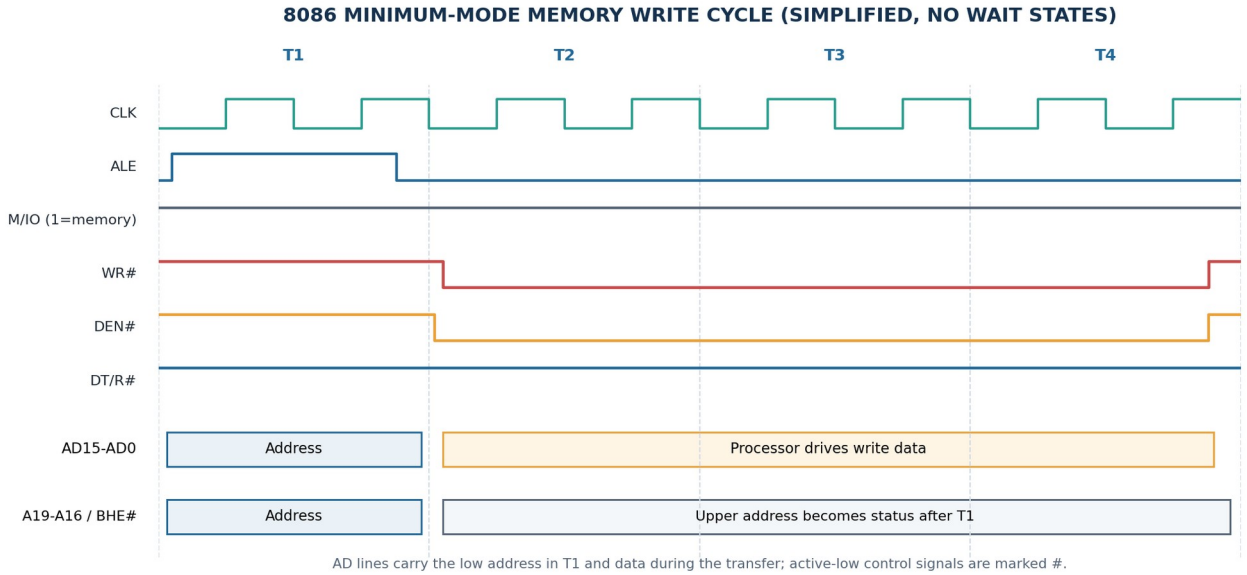


Figure 9. Simplified 8086 minimum-mode memory-write cycle across T1-T4.

The address is presented and latched in T1 as in the read cycle. For a write, DT/R# is high because the processor transmits data. The processor drives the data word or byte on AD15-AD0, DEN# enables the external transceiver, and WR# is asserted low to command the memory subsystem to accept the data. In the simplified waveform, the write command remains active into T4 to show the transfer window; exact control edges and required data hold depend on the specific 8086 timing specification and the external interface. A byte transfer also uses BHE# and A0 to select the high and/or low byte bank.

8.6 8085 and 8086 read/write comparison

Item	8085	8086 minimum mode
Typical no-wait memory cycle shown	T1, T2, T3	T1, T2, T3, T4
Multiplexed bus	AD7-AD0: low address then 8-bit data	AD15-AD0: low 16 address bits then 16-bit data
Address latch signal	ALE pulses during T1	ALE pulses during T1
Memory-cycle indication	IO/M = 0	M/IO = 1
Read command	RD# low in read transfer window	RD# low in read transfer window
Write command	WR# low in write transfer window	WR# low in write transfer window
Data width per bus transfer	8 bits	16 bits; A0 and BHE# determine byte-bank selection
Transceiver direction / enable	Depends on board-level interface; not represented by a single required 8085 DT/R# pin	DT/R# indicates transmit/receive direction; DEN# enables data transceivers in minimum mode
Wait states	May be added for slow devices	May be added for slow devices

8.7 Common errors when drawing the diagrams

- Leaving ALE high for the whole bus cycle. It is a pulse associated with the address phase, not a permanent indication that the bus contains an address.
- Forgetting the multiplexed-bus transition. After the address has been latched, the low-order address/data pins are reused for data.
- Drawing RD# or WR# active high. The # notation here explicitly means that the command is active when the line is low.
- Showing M/IO = 0 for an 8086 memory cycle. That is the 8085-style memory-cycle convention; for 8086 minimum mode M/IO is high for memory and low for I/O.
- Confusing a read with who drives the data bus. During a memory read, the memory subsystem supplies the data; during a memory write, the processor supplies it.
- Treating the pedagogical sketch as a pin-level substitute for the datasheet. Real timing must obey specified propagation delays, setup/hold windows, clock tolerances, and any wait-state requirements.

9. Conclusion

The design of a computer system can be understood at several connected levels. RISC and CISC describe contrasting instruction-set traditions, while Harvard and von Neumann describe how instruction and data memory paths are organized. Neither pair is a simple ranking: real implementations make trade-offs and often combine ideas from both traditions.

The 8085 illustrates a compact 8-bit processor with a 16-bit address space, an accumulator-centred ALU, register pairs, multiplexed low address/data pins, and a small set of control signals. The 8086 extends the datapath to 16 bits, expands the physical address space to 1 MiB through segment:offset addressing, introduces a BIU/EU division, and uses a 6-byte instruction prefetch queue. These differences affect programming, hardware interfacing, bus bandwidth, and timing.

At the logic level, adders implement arithmetic; encoders and decoders convert between line codes and binary representations; multiplexers route data; and flip-flops store state. The timing diagrams demonstrate how a processor and a memory device coordinate an address phase, control strobe, and data-transfer interval. Learning both the Boolean equations and the bus timing makes it easier to connect abstract instructions with physical digital hardware.

Appendix A. Quick Revision Sheet

Topic	Must-remember point
RISC / CISC	RISC tends toward regular, simple instructions and load/store; CISC tends toward richer, sometimes variable-length instructions. These are tendencies, not absolutes.
Harvard / von Neumann	Harvard: separate instruction/data paths. von Neumann: shared memory path. Modified Harvard is common in modern processors.
8085	8-bit data path and bus; 16-bit address bus; 64 KiB byte-address space; AD7-AD0 multiplexed; IO/M = 0 for memory.
8085 addressing	Immediate, register, direct, register-indirect (M means memory at HL), and implied/inherent.
8086	16-bit data bus; 20-bit address bus; 1 MiB physical address space; BIU/EU; 6-byte queue; segment:offset physical address.
8086 formula	Physical address = (segment x 16) + offset.
Half adder	S = A XOR B; C = AB.
Full adder	S = A XOR B XOR Cin; Cout = AB + ACin + BCin.
2:4 decoder	One of four outputs represents the two-bit input when enable is asserted.
4:1 multiplexer	S1S0 = 00 selects I0; 01 selects I1; 10 selects I2; 11 selects I3.
Flip-flops	SR: set/reset/hold plus forbidden basic case; JK: 11 toggles; D: Q(next)=D; T: 1 toggles, 0 holds.
Timing	Latch the multiplexed address during ALE; RD# is active-low read; WR# is active-low write; data source differs for reads and writes.

References

The following textbooks and processor manuals are suitable references for the concepts covered in this report. Signal-level timing and programming details should be checked against the relevant device documentation when designing or debugging an actual circuit.

1. Intel Corporation. 8085A / 8085AH Microprocessor data sheets and programming documentation. (Consult the specific manufacturer revision used by the laboratory or course.)
2. Intel Corporation. 8086 Family User's Manual. Intel, 1979. (Architecture, instruction set, memory segmentation, minimum-mode bus timing, and signal definitions.)
3. Gaonkar, Ramesh S. Microprocessor Architecture, Programming, and Applications with the 8085. (8085 architecture, instruction set, programming, and interfacing.)
4. Hall, Douglas V. Microprocessors and Interfacing: Programming and Hardware. (Microprocessor organization, bus signals, and interfacing principles.)
5. Mano, M. Morris, and Michael D. Ciletti. Digital Design. (Boolean logic, combinational circuits, sequential circuits, and flip-flops.)
6. Stallings, William. Computer Organization and Architecture: Designing for Performance. (Processor organization, instruction-set design, memory systems, and performance concepts.)

Note: This report uses simplified conceptual diagrams created for explanation. Timing figures intentionally omit wait states and are not drawn to scale; consult the datasheet for exact electrical timing values and system-specific variations.